

BSc (Hons) Artificial Intelligence and Data Science

Module: Artificial Intelligence (CM 2602)

Module Leader: Mr. Nipuna Senanayake

Individual Coursework Report

RGU Student ID : 2506755

IIT Student ID : 20240281

Student Name : W.D. Nisitha Nimsara

Contents

1.Introduction.....	1
2.Question - 1.....	2
2.1 Introduction.....	2
2.2 Assumptions.....	2
2.3 Decision Variables	2
2.4 Objective Function.....	2
2.5 Constraints	3
2.6 Implementation	4
2.6.1 Decision variable matrix	4
2.6.2 Score matrix	4
2.6.3 Capability table	4
2.6.4 Formulas used	5
2.6.5 Solver configurations	5
2.7 Results and Discussion	6
3.Question - 2.....	6
3.1 Introduction to Automotive Ontology	6
3.2 Scope of the Ontology	6
3.2.1 Knowledge Branches Covered.....	6
3.2.2 Competency Questions	7
3.3 Data Sources Used for Knowledge Gathering	7
3.4 Concept Graph / Taxonomy	8
3.5 Ontology Design (RDF/OWL)	8
3.6 SPARQL Queries, Results and Explanations	8
4.Question - 3.....	13
4.1 Introduction.....	13
4.2 Maze Representation.....	13
4.3 Algorithm Implementation.....	13
4.3.1 Iterative Deepening Depth-First Search (IDDFS)	13
4.3.2 Best First Search	14
4.4 Heuristic Function.....	14
4.5 Execution Results.....	14
4.6 Multiple Experiments.....	16
4.7 Alternative Heuristics	16
4.8 Performance Analysis	17
4.9 Statistical Analysis.....	17
4.10 Discussion	18

5.Question - 4.....	18
5.1 Introduction.....	18
5.2 Problem Definition and Objectives.....	18
5.3 Dataset and Experimental Setup	18
5.4 Machine Learning Approach	19
5.4.1 Model Selection	19
5.4.2 Training Process.....	20
5.4.3 Results.....	20
5.5 Fuzzy Logic Approach.....	21
5.5.1 Linguistic Variables and Membership Functions	21
5.5.2 Fuzzy Rule Base and Inference.....	22
5.5.3 Results.....	22
5.6 Rule-Based Approach	24
5.6.1 Threshold Design	24
5.6.2 Decision Logic	24
5.6.3 Results.....	24
5.7 Comparative Evaluation of the Three Approaches	26
5.8 Best Approach Selection and Justification	26
6.AI Usage Declaration.....	26
7.Appendix.....	27
8.Reference	27

List Of Figures

Figure 1 - CSP model.....	4
Figure 2 - Decision variable matrix	4
Figure 3 - Score matrix	4
Figure 4 - Capability table	4
Figure 5 - Solver configurations	5
Figure 7 - Concept graph of the automotive ontology	8
Figure 8 - SPARQL Query1	9
Figure 9 - SPARQL Query1 output	9
Figure 10 - SPARQL Query2	9
Figure 11 - SPARQL Query2 output	10
Figure 12 - SPARQL Query3	10
Figure 13 - SPARQL Query3 output	11
Figure 14 - SPARQL Queries4	11
Figure 15 - SPARQL Query4 output	11
Figure 16 - SPARQL Queries5	12
Figure 17 - SPARQL Query5 output	12
Figure 18 - IDDFS Pseudocode	13
Figure 19 - Best First Search Pseudocode	14
Figure 20 - Maze preview	14
Figure 21 - Record Information start, goal and barrier	15
Figure 22 - IDDFS Record Information.....	15
Figure 23 - BFS (Chebyshev) Record Information.....	15
Figure 24 - BFS (Manhattan) Record Information	16
Figure 25 - BFS (Euclidean) Record Information	16
Figure 26 - BFS (Diagonal) Record Information.....	16
Figure 27 - Comparison Table	17
Figure 28 - Dataset and Experimental Setup	19
Figure 29 - Machine Learning Approach.....	19
Figure 30 - Machine Learning overall result	20
Figure 31 - Machine Learning Severity Classification Report, severity confusion matrix and FP/FN summary.....	20
Figure 32 - Machine Learning Action Classification Report, Action confusion matrix and FP/FN summary.....	21
Figure 33 - Fuzzy Logic Membership Functions.....	21
Figure 34 - Fuzzy Logic Rules.....	22
Figure 35 - Fuzzy Logic overall results	22
Figure 36 - Fuzzy Logic Severity Classification Report, severity confusion matrix and FP/FN summary	23
Figure 37 - Fuzzy Logic Action Classification Report, Action confusion matrix and FP/FN summary	23
Figure 38 - Rule-based Decision Logics.....	24
Figure 39 - Rule-based overall results	24
Figure 40 - Rule-based Severity Classification Report, severity confusion matrix and FP/FN summary	25
Figure 41 - Rule-based Action Classification Report, Action confusion matrix and FP/FN summary.....	25
Figure 42 - Overall Comparative Evaluation.....	26

List of Tables

Table 1 - CSP Formulas.....	5
Table 2 - Data Sources and Usage table	7

1.Introduction

Artificial Intelligence (AI) plays a Important role in solving complex real-world problems by enabling systems to make intelligent decisions, represent knowledge effectively and learn from data. The coursework focuses on applying key concepts of AI such as Constraint Satisfaction Problems (CSP), Ontology Engineering, Search Algorithms, and Intelligent Decision Systems to practical scenarios.

The first part of this coursework models a real-world scheduling problem using CSP and solves it using Microsoft Excel Solver to optimize decision-making under constraints. The second part explores ontology engineering by designing a structured knowledge model for the automotive domain and querying it using SPARQL. The third part involves implementing and analyzing search algorithms to solve a path-finding problem in a maze environment, highlighting their efficiency and performance. Finally, the fourth part focuses on developing an anomaly detection and correction system for Automated Manual Transmission (AMT) systems using Machine Learning, Fuzzy Logic and Rule-Based approaches.

2.Question - 1

2.1 Introduction

This CSP question involves assigning 4 soldiers to a set of weekly patrol shifts such as day, night and boat patrols, while satisfying multiple operational constraints. The objective is to find feasible assignments while determining optimal solution that maximizes skill score by matching each patrol type with appropriately trained soldiers. This requires defining decision variables, constraints, and an objective function, followed by implementing the model in Excel and solving it using optimization techniques.

2.2 Assumptions

- Soldiers = {S1, S2, S3, S4}
- Patrol shifts:
 - Day Patrols = {D1, D2, D3, D4, D5}
 - Night Patrols = {N1, N2, N3, N4, N5}
 - Boat Patrols = {B1, B2, B3, B4}
- Total patrol shifts = 14
- Soldier capabilities:
 - Day-trained = { S1, S2}
 - Night-trained = { S3, S4}
 - Boat-certified = { S1, S3}

2.3 Decision Variables

Let:

$$x_{ij} = \begin{cases} 1, & \text{if soldier } i \text{ is assigned to patrol } j \\ 0, & \text{otherwise} \end{cases}$$

Where:

- $i \in \{S1, S2, S3, S4\}$
- $j \in \{D1 - D5, N1 - N5, B1 - B5\}$

2.4 Objective Function

$$\text{Maximize } Z = \sum_i \sum_j w_{ij} \cdot x_{ij}$$

Where:

- w_{ij} = Skill score

Scoring logic:

- 3 = Strong skill (trained)
- 1 = weak match

2.5 Constraints

1 - Minimum Assignment

Each soldier must be assigned to at least 2 patrols:

$$\sum_j x_{ij} \geq 2$$

2 - Maximum Assignment

Each soldier can be assigned to at most 5 patrols:

$$\sum_j x_{ij} \leq 5$$

3 - Total Assignment Limit

Fatigue Constraint:

$$\sum_i \sum_j x_{ij} \leq 14$$

4 - Patrol Coverage

Each patrol must have at least one soldier:

$$\sum_i x_{ij} \geq 1$$

5 - Night Patrol Requirement

Each night patrol must include at least one night-trained soldier (S3 or S4).

6 - Boat Patrol Requirement

Each boat patrol must include at least one boat-certified soldier (S1 or S3).

7 - Binary Constraint

$$x_{ij} \in \{0,1\}$$

2.6 Implementation

Complete excel sheet of CSP model:

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	Skill Score matrix															
2		D1	D2	D3	D4	D5	N1	N2	N3	N4	N5	B1	B2	B3	B4	
3	S1	3	3	3	3	3	3	1	1	1	1	1	3	3	3	3
4	S2	3	3	3	3	3	3	1	1	1	1	1	1	1	1	1
5	S3	1	1	1	1	1	1	3	3	3	3	3	3	3	3	3
6	S4	1	1	1	1	1	1	3	3	3	3	3	1	1	1	1
7																
8	Decision variables															
9		D1	D2	D3	D4	D5	N1	N2	N3	N4	N5	B1	B2	B3	B4	TotalAssigned
10	S1	1	0	0	0	1	0	0	0	0	0	0	1	0	0	1
11	S2	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0
12	S3	0	0	0	0	0	0	1	1	1	0	0	0	1	1	0
13	S4	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
14	PatrolCoverage	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
15																
16	NightCoverage						1	1	1	1	1					
17	BoatCoverage											1	1	1	1	
18																
19	Objective	42					DaySkill	NightSkill	BoatSkill							
20							S1	1	0	1						
21							S2	1	0	0						
22							S3	0	1	1						
23							S4	0	1	0						
24																

Figure 1 - CSP model

2.6.1 Decision variable matrix

		D1	D2	D3	D4	D5	N1	N2	N3	N4	N5	B1	B2	B3	B4	TotalAssigned
10	S1	1	0	0	0	1	0	0	0	0	0	0	1	0	0	1
11	S2	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0
12	S3	0	0	0	0	0	0	1	1	1	0	0	0	1	1	0
13	S4	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
14	PatrolCoverage	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Figure 2 - Decision variable matrix

2.6.2 Score matrix

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	Skill Score matrix														
2		D1	D2	D3	D4	D5	N1	N2	N3	N4	N5	B1	B2	B3	B4
3	S1	3	3	3	3	3	1	1	1	1	1	3	3	3	3
4	S2	3	3	3	3	3	1	1	1	1	1	1	1	1	1
5	S3	1	1	1	1	1	3	3	3	3	3	3	3	3	3
6	S4	1	1	1	1	1	3	3	3	3	3	1	1	1	1

Figure 3 - Score matrix

2.6.3 Capability table

	DaySkill	NightSkill	BoatSkill
S1	1	0	1
S2	1	0	0
S3	0	1	1
S4	0	1	0

Figure 4 - Capability table

2.6.4 Formulas used

Purpose	Cell(s)	Formula	Explanation
Total patrols assigned to S1	P10	=SUM(B10:O10)	Counts all assignments in S1's decision row.
Total patrols assigned to S2	P11	=SUM(B11:O11)	Counts all assignments in S2's decision row.
Total patrols assigned to S3	P12	=SUM(B12:O12)	Counts all assignments in S3's decision row.
Total patrols assigned to S4	P13	=SUM(B13:O13)	Counts all assignments in S4's decision row.
Coverage of each patrol	B14:O14	In B14 use =SUM(B10:B13) and copy across to O14	Checks that every patrol has at least one soldier.
Grand total assignments	P14	=SUM(P10:P13)	Checks the overall fatigue limit of 14.
Night Coverage	G16:K16	In G16 use =SUM(G10:G13) and copy across to K16	Check night petrol has night-trained soldiers.
Boat Coverage	L17:O17	In L17 use =SUM(L10:L13) and copy across to O17	Check boat petrol has boat-certified soldiers.
Objective function	B19	=SUMPRODUCT(B3:O6,B10:O13)	Multiplies Skill scores by assignment decisions and sums the total mission score.

Table 1 - CSP Formulas

2.6.5 Solver configurations

Solver Parameters

Set Objective:

To: ☒ Max ☐ Min ☐ Value Of:

By Changing Variable Cells:

Subject to the Constraints:

☒ Make Unconstrained Variables Non-Negative

Select a Solving Method:

Options

Solving Method

Select the GRG Nonlinear engine for Solver Problems that are smooth nonlinear. Select the LP Simplex engine for linear Solver Problems, and select the Evolutionary engine for Solver problems that are non-smooth.

Figure 5 - Solver configurations

2.7 Results and Discussion

The solution is:

- S1 assigned to D1, D4, B1 and B4
- S2 assigned to D2, D3 and D5
- S3 assigned to N1, N2, N3, B2 and B3
- S4 assigned to N4 and N5

This gives the soldiers to 4, 3, 5, and 2 assignments, respectively, for a total of 14 assignments. Each patrol column has coverage equal to 1. Night coverage equals 1 for the five night patrols, and boat coverage equals 1 for each of the four boat patrols.

The objective value is 42. This is the maximum possible value since there are 14 patrols and each patrol is perfectly matched with a suitable soldier, giving the highest score of 3 for each patrol assigned. Therefore, the solution is not only feasible but also optimal within the chosen scoring model.

3.Question - 2

3.1 Introduction to Automotive Ontology

Ontology is a structured way of representing knowledge in a machine-readable form. In the automotive domain, it helps organize information about vehicle parts, manufacturing steps, testing methods, standards and maintenance activities. This makes the knowledge easier to understand, connect, and query.

3.2 Scope of the Ontology

This ontology focuses on five main areas of the automotive domain:

1. Vehicle Components
2. Manufacturing Processes
3. Quality Assurance and Testing
4. Compliance and Regulations
5. Maintenance and Security Policies

These areas were selected because they represent important parts of modern vehicle development and production. The model is intended for engineers, production staff, quality teams and researchers who need a structured view of automotive knowledge.

3.2.1 Knowledge Branches Covered

The ontology was organized into five main branches.

- Vehicle Components branch covers major component categories used in a vehicle, including powertrain parts, chassis and safety parts, and electrical/electronic parts.
- Manufacturing Processes branch covers body manufacturing, coating and assembly, and inspection activities used during automotive production.
- Quality Assurance and Testing branch covers quality core tools and testing or validation methods used to ensure product quality.

- Compliance and Regulations branch covers automotive standards and regulations that influence safety, quality, and cybersecurity.
- Maintenance and Security Policies branch covers maintenance-related policies and security-related control policies that support vehicle reliability and protection.

3.2.2 Competency Questions

The ontology was designed to answer the following questions:

1. Which powertrain components are manufactured by which manufacturing processes?
2. Which vehicle components are validated by which quality assurance or testing entities?
3. Which vehicle components comply with which automotive standards or regulations?
4. Which electrical and electronic components are protected by which security control policies?
5. Which vehicle components are maintained by which maintenance policies?

3.3 Data Sources Used for Knowledge Gathering

No.	Source	Accessible link	Usage
1	NHTSA	NHTSA Statutes, Regulations, Authorities & FMVSS	Used for the Compliance and Regulations branch
2	AIAG	Quality Core Tools	Used for the Quality Assurance and Testing branch.
3	UNECE	UN Regulation No. 155 – Cyber Security and Cyber Security Management System	Used for the Maintenance and Security Policies and Compliance branches.
4	Monolithic Power Systems	Basics of Automotive Electronic Systems	Used for the Vehicle Components branch.
5	Automotive coating review	Review of Coating and Curing Processes in Automotive Industry (PDF)	Used for the Manufacturing Processes branch.

Table 2 - Data Sources and Usage table

3.4 Concept Graph / Taxonomy

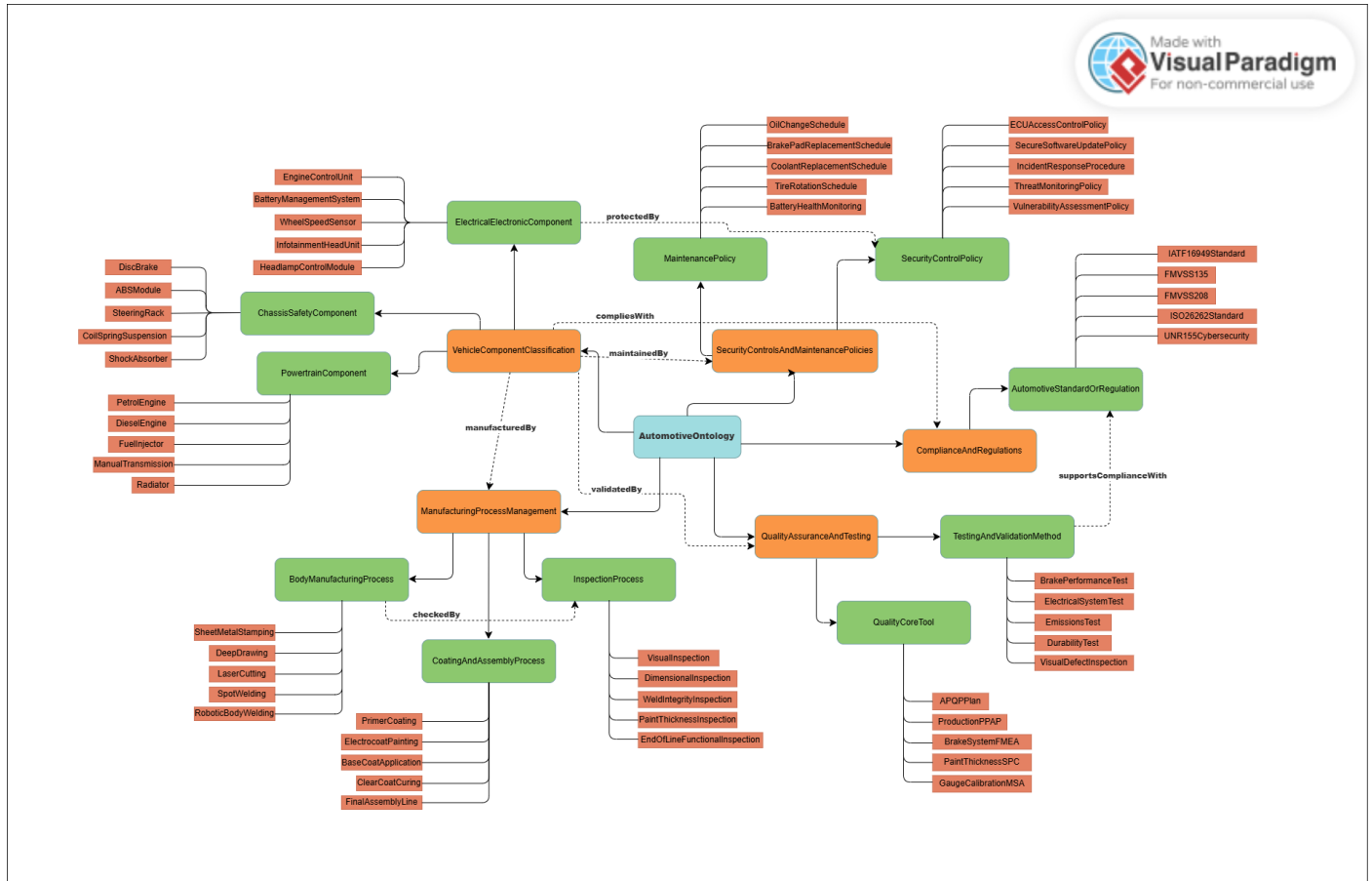


Figure 6 - Concept graph of the automotive ontology

3.5 Ontology Design (RDF/OWL)

The ontology graph was implemented using RDF/OWL syntax. so that the concept graph becomes machine-readable. The code is written in the file named Question2Automotive.rdf. The source code is submitted along with this report.

3.6 SPARQL Queries, Results and Explanations

When the questions are given, SPARQL is used to query the ontology and answer the questions. Each query was written to retrieve useful information.

1. Which powertrain components are manufactured by which manufacturing processes?

Query:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX auto: <http://www.example.org/automotive#>

SELECT ?componentLabel ?processLabel
WHERE {
    ?component rdf:type auto:PowertrainComponent ;
        rdfs:label ?componentLabel ;
        auto:manufacturedBy ?process .
```

```

?process rdfs:label ?processLabel .
}

```

Figure 7 - SPARQL Query1

This SPARQL query retrieves all individuals that belong to the PowertrainComponent class and identifies the manufacturing process linked to each component through the manufacturedBy object property. The query uses the component label and process label to display human-readable results instead of URI references.

Result:

componentLabel	processLabel
Radiator	FinalAssemblyLine
ManualTransmission	FinalAssemblyLine
FuelInjector	LaserCutting
DieselEngine	FinalAssemblyLine
PetrolEngine	FinalAssemblyLine

Figure 8 - SPARQL Query1 output

The result shows that most of the powertrain components in the ontology, namely DieselEngine, ManualTransmission, PetrolEngine, and Radiator, are associated with the FinalAssemblyLine manufacturing process. In contrast, the FuelInjector is linked to LaserCutting, indicating that it is manufactured using a more specialized production step.

2. Which vehicle components are validated by which quality assurance or testing entities?

Query:

```

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX auto: <http://www.example.org/automotive#>

SELECT ?componentLabel ?validatorLabel
WHERE {
  {
    ?component rdf:type auto:PowertrainComponent .
  }
  UNION
  {
    ?component rdf:type auto:ChassisSafetyComponent .
  }
  UNION
  {
    ?component rdf:type auto:ElectricalElectronicComponent .
  }

  ?component rdfs:label ?componentLabel ;
    auto:validatedBy ?validator .

  ?validator rdfs:label ?validatorLabel .
}

```

Figure 9 - SPARQL Query2

This SPARQL query retrieves vehicle components from the three main component categories in the ontology: PowertrainComponent, ChassisSafetyComponent and ElectricalElectronicComponent. It then identifies the quality assurance tool or testing method connected to each component through the validatedBy object property. The use of UNION allows the query to combine results from multiple subclasses into one single output table.

Result:

componentLabel	validatorLabel
ManualTransmission	DurabilityTest
FuelInjector	APQPPlan
DieselEngine	ProductionPPAP
PetrolEngine	ProductionPPAP
ShockAbsorber	DurabilityTest
CoilSpringSuspension	DurabilityTest
ABSModule	BrakePerformanceTest
DiscBrake	BrakeSystemFMEA
DiscBrake	BrakePerformanceTest
WheelSpeedSensor	ElectricalSystemTest
BatteryManagementSystem	ElectricalSystemTest
EngineControlUnit	APQPPlan
EngineControlUnit	ElectricalSystemTest

Figure 10 - SPARQL Query2 output

The result shows that different automotive components are validated using different testing methods and quality tools depending on their function.

3. Which vehicle components comply with which automotive standards or regulations?

Query:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX auto: <http://www.example.org/automotive#>

SELECT ?componentLabel ?standardLabel
WHERE {
  {
    ?component rdf:type auto:PowertrainComponent .
  }
  UNION
  {
    ?component rdf:type auto:ChassisSafetyComponent .
  }
  UNION
  {
    ?component rdf:type auto:ElectricalElectronicComponent .
  }

  ?component rdfs:label ?componentLabel ;
    auto:compliesWith ?standard .

  ?standard rdfs:label ?standardLabel .
}
```

Figure 11 - SPARQL Query3

This SPARQL query retrieves all vehicle components from the three major ontology branches: PowertrainComponent, ChassisSafetyComponent and ElectricalElectronicComponent. It then identifies the automotive standard or regulation connected to each component through the compliesWith object property. The query helps reveal how the ontology models compliance relationships between physical or electronic automotive parts and the standards that govern them.

Result:

componentLabel	standardLabel
DieselEngine	IATF16949Standard
PetrolEngine	IATF16949Standard
ABSModule	FMVSS135
DiscBrake	FMVSS135
HeadlampControlModule	FMVSS208
BatteryManagementSystem	ISO26262Standard
EngineControlUnit	UNR155Cybersecurity
EngineControlUnit	ISO26262Standard

Figure 12 - SPARQL Query3 output

The result shows that different vehicle components are linked to different regulatory or quality standards depending on their role in the vehicle system.

4. Which electrical and electronic components are protected by which security control policies?

Query:

PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX auto: <http://www.example.org/automotive#>
SELECT ?componentLabel ?policyLabel
WHERE {
?component rdf:type auto:ElectricalElectronicComponent ;
rdfs:label ?componentLabel ;
auto:protectedBy ?policy .
?policy rdfs:label ?policyLabel .
}

Figure 13 - SPARQL Queries4

This SPARQL query retrieves all individuals that belong to the ElectricalElectronicComponent class and identifies the security control policy associated with each component through the protectedBy object property. The query uses readable labels for both the component and the policy so that the output clearly shows how cybersecurity protections are assigned within the ontology.

Result:

componentLabel	policyLabel
InfotainmentHeadUnit	VulnerabilityAssessmentPolicy
WheelSpeedSensor	ThreatMonitoringPolicy
EngineControlUnit	SecureSoftwareUpdatePolicy
EngineControlUnit	ECUAccessControlPolicy

Figure 14 - SPARQL Query4 output

The result shows that several electrical and electronic components are linked to specific security control policies.

5. Which vehicle components are maintained by which maintenance policies?

Query:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX auto: <http://www.example.org/automotive#>

SELECT ?componentLabel ?policyLabel
WHERE {
  {
    ?component rdf:type auto:PowertrainComponent .
  }
  UNION
  {
    ?component rdf:type auto:ChassisSafetyComponent .
  }
  UNION
  {
    ?component rdf:type auto:ElectricalElectronicComponent .
  }

  ?component rdfs:label ?componentLabel ;
    auto:maintainedBy ?policy .

  ?policy rdf:type auto:MaintenancePolicy ;
    rdfs:label ?policyLabel .
}
```

Figure 15 - SPARQL Queries5

This SPARQL query retrieves vehicle components from the three major component categories in the ontology and identifies the maintenance policy associated with each one through the maintainedBy object property. The query also restricts the linked resource to the MaintenancePolicy class, ensuring that only maintenance-related policies are returned in the result set. In this way, the query clearly shows how the ontology models maintenance requirements for selected automotive components.

Result:

componentLabel	policyLabel
Radiator	CoolantReplacementSchedule
DieselEngine	OilChangeSchedule
PetrolEngine	OilChangeSchedule
DiscBrake	BrakePadReplacementSchedule
BatteryManagementSystem	BatteryHealthMonitoring

Figure 16 - SPARQL Query5 output

The result shows that different vehicle components are linked to specific maintenance policies according to their operational needs.

4.Question - 3

4.1 Introduction

The question is to find a valid path in a maze from start node to goal node while avoiding obstacles. The maze is represented as a 6×6 grid where each cell corresponds to a node. Some nodes are blocked (barriers) and the agent can move in eight possible directions.

The main objective of is to compare how different search strategies perform in solving this problem. Two types of algorithms were implemented:

- Iterative Deepening Depth-First Search (IDDFS) - an uninformed search algorithm
- Best First Search - an informed search algorithm that uses heuristic functions

Multiple heuristic functions were tested to evaluate how the choice of heuristic affects performance.

4.2 Maze Representation

The maze is modeled as a grid of 36 nodes indexed from 0 to 35. Each node corresponds to a coordinate in the 6×6 grid. To efficiently represent the maze and support search operations, the following data structures were used:

- Dictionary: to store the maze configuration (start node, goal node, and barriers)
- Set: to store barrier nodes for fast lookup during traversal
- List: to track visited nodes and paths
- priority queue (heap): for Best First Search to always expand the most promising node

Each node can have up to eight neighbors depending on its position and whether surrounding nodes are blocked.

4.3 Algorithm Implementation

4.3.1 Iterative Deepening Depth-First Search (IDDFS)

IDDFS is a combination of Depth-First Search (DFS) and Breadth-First Search (BFS). It works by repeatedly performing depth-limited searches with increasing depth limits until the goal is found.

At each iteration:

- The algorithm explores nodes up to a certain depth
- If the goal is not found, the depth limit is increased
- The process repeats until a solution is discovered

This approach ensures completeness while avoiding the memory overhead of BFS.

Pseudocode

```
for depth = 0 to max_depth:
    result = depth_limited_search(start, depth)
    if result is found:
        return result
```

Figure 17 - IDDFS Pseudocode

4.3.2 Best First Search

Best First Search is an informed search algorithm that uses a heuristic function to decide which node to explore next. Instead of exploring blindly, it always selects the node that appears closest to the goal. The algorithm uses a priority queue:

- Nodes are prioritized based on their heuristic value
- The node with the lowest estimated distance to the goal is expanded first

This approach makes the search process significantly faster compared to uninformed methods.

Pseudocode

```
initialize priority queue
add start node

while queue is not empty:
    select node with lowest heuristic value
    if node is goal:
        return path
    expand its neighbors
```

Figure 18 - Best First Search Pseudocode

4.4 Heuristic Function

The initial heuristic was Chebyshev distance, which is suitable for movement in eight directions.

Chebyshev Distance:

$$h(n) = \max(|x1 - x2|, |y1 - y2|)$$

This estimates the number of steps required to reach the goal when diagonal movement is allowed.

Later, additional heuristics were introduced to compare performance:

- Manhattan distance
- Euclidean distance
- Diagonal distance

These heuristics provide different ways of estimating the distance to the goal.

4.5 Execution Results

6x6 Maze:

X	6	12	18	24	30
1	7	13	19	25	31
2	8	14	20	26	32
3	5	15	21	27	33
4	10	16	22	G	34
X	X	X	23	29	35

Figure 19 - Maze preview

For each run, the following information was recorded:

- Start node and its coordinates
- Goal node and its coordinates
- Barrier positions

```
Start node: 9 Coordinates: (1, 3)
Goal node: 28 Coordinates: (4, 4)
Barrier nodes: [0, 5, 11, 17]
```

Figure 20 - Record Information start, goal and barrier

- Order of visited nodes
- Number of visited nodes
- Time taken
- Final path from start to goal
- Path length
- Path cost

```
--- IDDFS ---
Visited nodes: [9, 9, 2, 3, 4, 8, 10, 14, 15, 16, 9, 2, 1, 3, 7, 8, 3, 2, 4, 8, 10, 4, 3, 10, 8, 1,
2, 3, 7, 13, 14, 15, 10, 3, 4, 15, 16, 14, 7, 8, 13, 15, 19, 20, 21, 15, 8, 10, 14, 16, 20, 21, 22
, 16, 10, 15, 21, 22, 23, 9, 2, 1, 6, 7, 8, 3, 4, 8, 10, 7, 1, 6, 8, 12, 13, 14, 8, 1, 3, 7, 13, 14
, 15, 3, 2, 1, 7, 8, 4, 10, 8, 1, 2, 7, 13, 14, 15, 10, 4, 15, 16, 4, 3, 2, 8, 10, 10, 3, 15, 16, 8
, 1, 2, 6, 7, 2, 1, 3, 7, 3, 2, 4, 10, 7, 1, 2, 6, 12, 13, 14, 13, 6, 7, 12, 14, 18, 19, 20, 14, 7,
13, 15, 19, 20, 21, 15, 10, 14, 16, 20, 21, 22, 10, 3, 2, 4, 8, 4, 3, 15, 8, 14, 16, 20, 21, 22, 1
6, 15, 21, 22, 23, 14, 7, 1, 2, 6, 8, 12, 13, 8, 1, 2, 3, 7, 13, 15, 13, 6, 7, 8, 12, 18, 19, 20, 1
5, 8, 10, 16, 20, 21, 22, 19, 12, 13, 18, 20, 24, 25, 26, 20, 13, 15, 19, 21, 25, 26, 27, 21, 15, 1
6, 20, 22, 26, 27, 28]
Number of visited nodes: 225
Time to find goal: 225 minutes
Final path: [9, 14, 21, 28]
Path length: 4
Path cost: 4.242640687119286
```

Figure 21 - IDDFS Record Information

```
--- Best First (Chebyshev) ---
Visited nodes: [9, 14, 21, 28]
Number of visited nodes: 4
Time to find goal: 4 minutes
Final path: [9, 14, 21, 28]
Path length: 4
Path cost: 4.242640687119286
```

Figure 22 - BFS (Chebyshev) Record Information

```
--- Best First (Manhattan) ---  
Visited nodes: [9, 16, 22, 28]  
Number of visited nodes: 4  
Time to find goal: 4 minutes  
Final path: [9, 16, 22, 28]  
Path length: 4  
Path cost: 3.414213562373095
```

Figure 23 - BFS (Manhattan) Record Information

```
--- Best First (Euclidean) ---  
Visited nodes: [9, 16, 22, 28]  
Number of visited nodes: 4  
Time to find goal: 4 minutes  
Final path: [9, 16, 22, 28]  
Path length: 4  
Path cost: 3.414213562373095
```

Figure 24 - BFS (Euclidean) Record Information

```
--- Best First (Diagonal) ---  
Visited nodes: [9, 16, 22, 28]  
Number of visited nodes: 4  
Time to find goal: 4 minutes  
Final path: [9, 16, 22, 28]  
Path length: 4  
Path cost: 3.414213562373095
```

Figure 25 - BFS (Diagonal) Record Information

4.6 Multiple Experiments

To ensure that the results are not biased by a single maze configuration, the experiment was repeated three times with different randomly generated mazes and all above information were records on each maze. This allowed a more reliable comparison between algorithms by analysing average performance rather than a single outcome.

4.7 Alternative Heuristics

In addition to Chebyshev distance, three other heuristic functions were evaluated:

Manhattan Distance:

$$h(n) = (|x1 - x2| + |y1 - y2|)$$

Euclidean Distance:

$$h(n) = \text{sqrt}((x1 - x2)^2 + (y1 - y2)^2)$$

Diagonal Distance:

$$h(n) = \max(dx, dy) + (\sqrt{2} - 1) * \min(dx, dy)$$

Each heuristic estimates the distance differently, which affects how the search algorithm prioritizes nodes.

4.8 Performance Analysis

Completeness

- IDDFS is a complete algorithm, meaning it is guaranteed to find a solution if one exists. This is because it systematically explores all possible paths with increasing depth.
- Best First Search on the other hand, is not always guaranteed to find a solution in all cases. However, in this implementation, it successfully found valid paths in all test cases.

Optimality

- IDDFS can produce optimal solutions, if all step costs are equal. However, it is inefficient because it repeatedly explores the same nodes at different depth levels.
- Best First Search is not guaranteed to produce the optimal path because it relies solely on heuristic values. However, depending on the heuristic used, it can produce near-optimal solutions.

Time Complexity

- IDDFS has high time complexity due to repeated depth-limited searches. As a result, it expands large number of nodes.
- Best First Search is comparatively more efficient because it uses heuristic guidance to focus the search towards the goal, reducing unnecessary exploration.

4.9 Statistical Analysis

COMPARISON TABLE				
Algorithm	Mean Time	Var Time	Mean Len	Mean Cost
IDDFS	1124.0000	981307.0000	5.0000	5.3807
Chebyshev	5.0000	1.0000	5.0000	5.3807
Manhattan	5.0000	1.0000	5.0000	4.8284
Euclidean	5.0000	1.0000	5.0000	4.8284
Diagonal	5.0000	1.0000	5.0000	4.8284

Figure 26 - Comparison Table

Time Analysis

IDDFS showed a very high average time compared to Best First Search. This confirms that uninformed search is inefficient for this problem.

All heuristic-based searches had similar average times, indicating that heuristic choice does not significantly impact execution speed in this case.

Path Length Analysis

All algorithms produced similar path lengths on average. This suggests that both uninformed and informed methods are capable of finding reasonably short paths.

Path Cost Analysis

A clear difference was observed in path cost, Chebyshev and IDDFS produced slightly higher costs while Manhattan, Euclidean, and Diagonal produced lower costs, This indicates that these heuristics provide better guidance towards the goal, resulting in more efficient paths.

4.10 Discussion

From the results, it is clear that **Best First Search is significantly more efficient than IDDFS** in terms of time. IDDFS explores a large number of nodes due to its uninformed nature, making it impractical for larger search spaces.

Although all heuristic variants of Best First Search performed similarly in terms of speed, they differed in the quality of the paths produced. Manhattan, Euclidean and Diagonal heuristics consistently resulted in lower path costs, indicating better performance.

Overall, Best First Search with an appropriate heuristic (such as Manhattan, Euclidean or Diagonal distance) provides the best balance between efficiency and solution quality for this type of grid-based problem.

5.Question - 4

5.1 Introduction

Automated Manual Transmission (AMT) systems are widely used in modern vehicles to provide a balance between driving comfort, fuel efficiency and mechanical control. Since AMT operation depends on coordinated sensor readings and actuator responses, faults such as delayed gear engagement, clutch overheating, torque imbalance and engine RPM mismatch can reduce transmission stability and increase mechanical wear. Therefore, an intelligent anomaly detection and correction mechanism is important for identifying abnormal operating conditions early and recommending suitable recovery actions before the fault becomes more severe. This question is about developing such a system for AMT operations using three different artificial intelligence approaches: Machine Learning, Fuzzy Logic and a simple Rule-Based method. The final aim is to detect anomaly severity and correction action then determine which method is most suitable for real-time AMT deployment.

5.2 Problem Definition and Objectives

The AMT anomaly detection is defined using four input variables: gear engagement delay, clutch temperature variation, torque fluctuation and engine RPM mismatch. Based on these sensor inputs, each approach must perform two tasks. First, it must classify the anomaly severity as Normal, Moderate, or Severe. Second, it must recommend appropriate corrective action. The actions are NO_ACTION, SHIFT_TIMING_ADJUST, CLUTCH_PRESSURE_RECALIB, TORQUE_REDISTRIBUTION and RPM_SYNC. The overall objective is to compare the three approaches in terms of anomaly detection quality and practical deployment suitability, especially with respect to inference time and mitigation effectiveness.

5.3 Dataset and Experimental Setup

The AMT anomaly dataset contains 60 records in total. The severity distribution of the dataset is 26 Moderate, 22 Severe and 12 Normal samples. The action distribution is 26 TORQUE_REDISTRIBUTION, 12 NO_ACTION, 10 SHIFT_TIMING_ADJUST, 6 CLUTCH_PRESSURE_RECALIB and 6 RPM_SYNC samples. To ensure a fair comparison, the same stratified train/test split was used for all three approaches. The dataset was divided into 48 training records and 12 testing records, and the code used a combined stratification

label to preserve both severity and action proportions across the split. This is an important design decision because it avoids giving one method an easier or harder test set than the others.

```
Features used: ['delay_ms', 'clutch_temp_c', 'torque_var_pct', 'rpm_diff']
Severity distribution: {'Moderate': 26, 'Severe': 22, 'Normal': 12}
Action distribution:  {'TORQUE_REDISTRIBUTION': 26, 'NO_ACTION': 12, 'SHIFT_TIMING_ADJUST': 10, 'CLUTCH_PRESSURE_RECALIB': 6, 'RPM_SYNC': 6}

Train records: 48 | Test records: 12
All three approaches below are evaluated on the SAME test set.
```

Figure 27 - Dataset and Experimental Setup

5.4 Machine Learning Approach

5.4.1 Model Selection

For the Machine Learning approach, two Random Forest classifiers were used: one for severity prediction and one for correction action prediction. Label encoding was first applied to convert the categorical output labels into numerical form. Random Forest was selected because it can model non-linear relationships between sensor features and output classes while remaining relatively efficient at prediction time. In this implementation, both classifiers were configured with balanced class weights in order to reduce the effect of class imbalance.

```
# -----
# APPROACH 1 - MACHINE LEARNING
def build_ml_models(X_train: np.ndarray, y_train_sev: np.ndarray, y_train_act: np.ndarray): 1 usage
    sev_encoder = LabelEncoder()
    act_encoder = LabelEncoder()

    y_train_sev_enc = sev_encoder.fit_transform(y_train_sev)
    y_train_act_enc = act_encoder.fit_transform(y_train_act)

    rf_sev = RandomForestClassifier(
        n_estimators=100,
        random_state=RANDOM_STATE,
        class_weight="balanced",
        min_samples_leaf=1)
    rf_act = RandomForestClassifier(
        n_estimators=100,
        random_state=RANDOM_STATE,
        class_weight="balanced",
        min_samples_leaf=1)

    rf_sev.fit(X_train, y_train_sev_enc)
    rf_act.fit(X_train, y_train_act_enc)

    def predict_fn(X_batch: np.ndarray) -> Tuple[List[str], List[str]]:
        pred_sev = sev_encoder.inverse_transform(rf_sev.predict(X_batch)).tolist()
        pred_act = act_encoder.inverse_transform(rf_act.predict(X_batch)).tolist()
        return pred_sev, pred_act

    return predict_fn
```

Figure 28 - Machine Learning Approach

5.4.2 Training Process

Models were trained using the same feature matrix derived from the four AMT sensor variables. One model learned the mapping from sensor values to anomaly severity, while the second model learned the mapping from sensor values to corrective action. The trained models were then evaluated on the common test set of 12 records. Since all three approaches were tested on the same data partition, the performance comparison remained fair and directly comparable.

5.4.3 Results

The Machine Learning approach achieved 75.00% severity accuracy and 91.67% action accuracy, with an average inference time of 3.582 ms for the 12-record test batch, corresponding to 0.2985 ms per record. For severity prediction, the method performed particularly well on the Normal class, where recall reached 1.00, and it also maintained strong performance on the Moderate class. However, the model still produced two false negatives for the Severe class, meaning two severe anomalies were predicted as lower severity levels. For action prediction, the model was highly accurate, correctly identifying all instances of SHIFT_TIMING_ADJUST, CLUTCH_PRESSURE_RECALIB and RPM_SYNC, while making only one false negative on TORQUE_REDISTRIBUTION. These results show that the Machine Learning method offers a strong balance between detection quality and action recommendation quality.

```
Inference time (average for 12 test records): 3.582 ms
Per-record inference time: 0.2985 ms
Severity accuracy: 75.00%
Action accuracy: 91.67%
```

Figure 29 - Machine Learning overall result

-- Severity Classification Report (test set) --				
	precision	recall	f1-score	support
Normal	0.67	1.00	0.80	2
Moderate	0.67	0.80	0.73	5
Severe	1.00	0.60	0.75	5
accuracy			0.75	12
macro avg	0.78	0.80	0.76	12
weighted avg	0.81	0.75	0.75	12
Severity Confusion Matrix				
	Pred_Normal	Pred_Moderate	Pred_Severe	
Actual_Normal	2	0	0	
Actual_Moderate	1	4	0	
Actual_Severe	0	2	3	
Severity FP/FN Summary				
class	TP	FP	FN	TN
Normal	2	1	0	9
Moderate	4	2	1	5
Severe	3	0	2	7

Figure 30 - Machine Learning Severity Classification Report, severity confusion matrix and FP/FN summary

```
-- Action Classification Report (test set) --
              precision    recall  f1-score   support

   NO_ACTION      0.67      1.00      0.80         2
  SHIFT_TIMING_ADJUST  1.00      1.00      1.00         2
 CLUTCH_PRESSURE_RECALIB  1.00      1.00      1.00         1
  TORQUE_REDISTRIBUTION  1.00      0.83      0.91         6
      RPM_SYNC      1.00      1.00      1.00         1

 accuracy              0.92      12
  macro avg           0.93      0.97      0.94      12
  weighted avg           0.94      0.92      0.92      12
```

Action Confusion Matrix

	Pred_NO_ACTION	Pred_SHIFT_TIMING_ADJUST	Pred_CLUTCH_PRESSURE_RECALIB	Pred_TORQUE_REDISTRIBUTION	Pred_RPM_SYNC
Actual_NO_ACTION	2	0	0	0	0
Actual_SHIFT_TIMING_ADJUST	0	2	0	0	0
Actual_CLUTCH_PRESSURE_RECALIB	0	0	1	0	0
Actual_TORQUE_REDISTRIBUTION	1	0	0	5	0
Actual_RPM_SYNC	0	0	0	0	1

Action FP/FN Summary

class	TP	FP	FN	TN
NO_ACTION	2	1	0	9
SHIFT_TIMING_ADJUST	2	0	0	10
CLUTCH_PRESSURE_RECALIB	1	0	0	11
TORQUE_REDISTRIBUTION	5	0	1	6
RPM_SYNC	1	0	0	11

Figure 31 - Machine Learning Action Classification Report, Action confusion matrix and FP/FN summary

5.5 Fuzzy Logic Approach

5.5.1 Linguistic Variables and Membership Functions

The Fuzzy Logic system was designed using the same four AMT features as antecedent variables. For each feature, the code automatically constructed low, medium and high membership functions using the training data distribution. The output variable, severity_out, was represented on a scale from 0 to 10 and mapped into fuzzy sets corresponding to normal, moderate and severe. This design allows the system to express gradual transitions between operating states rather than using abrupt boundaries.

```
283 def add_memberships(var, feature_name: str):
284     s_all = train_df[feature_name]
285     s_n = normal_df[feature_name]
286     s_m = moderate_df[feature_name]
287     s_s = severe_df[feature_name]
288
289     all_min = float(s_all.min())
290     all_max = float(s_all.max())
291     low_end = _safe_quantile(s_n, q: 0.75, s_all.quantile(0.35))
292     mid_left = _safe_quantile(s_m, q: 0.25, s_all.quantile(0.45))
293     mid_peak = _safe_median(s_m, s_all.quantile(0.55))
294     mid_right = _safe_quantile(s_m, q: 0.75, s_all.quantile(0.70))
295     high_start = _safe_quantile(s_s, q: 0.25, s_all.quantile(0.75))
296
297     var["low"] = fuzz.trapmf(var.universe, _ordered4(all_min, all_min, low_end, mid_left))
298     var["medium"] = fuzz.trimf(var.universe, _ordered3(mid_left, mid_peak, high_start))
299     var["high"] = fuzz.trapmf(var.universe, _ordered4(mid_right, high_start, all_max, all_max))
300
301     add_memberships(delay, feature_name: "delay_ms")
302     add_memberships(temp, feature_name: "clutch_temp_c")
303     add_memberships(torque, feature_name: "torque_var_pct")
304     add_memberships(rpm, feature_name: "rpm_diff")
305
306     severity_out["normal"] = fuzz.trimf(severity_range, abc: [0, 0, 4])
307     severity_out["moderate"] = fuzz.trimf(severity_range, abc: [2.5, 5, 7.5])
308     severity_out["severe"] = fuzz.trimf(severity_range, abc: [6, 10, 10])
309
```

Figure 32 - Fuzzy Logic Membership Functions

5.5.2 Fuzzy Rule Base and Inference

The fuzzy rule base was built around four membership values such as delay, temp, torque and rpm. For example, when all input variables were in the low range, the system inferred normal severity. If any variable entered the medium range, the system could infer moderate severity, while high values led to severe severity. Additional rules were also included to upgrade combined medium-level conditions to severe, such as delay medium AND temp medium then severe. After fuzzy inference, the resulting numeric score was converted into one of the three severity classes using score thresholds. The corrective action was then chosen based on the feature that deviated most strongly from its learned thresholds.

```
310  ~    rules = [  
311      ctrl.Rule(delay["low"] & temp["low"] & torque["low"] & rpm["low"], severity_out["normal"]),  
312      ctrl.Rule(delay["medium"], severity_out["moderate"]),  
313      ctrl.Rule(temp["medium"], severity_out["moderate"]),  
314      ctrl.Rule(torque["medium"], severity_out["moderate"]),  
315      ctrl.Rule(rpm["medium"], severity_out["moderate"]),  
316      ctrl.Rule(delay["high"], severity_out["severe"]),  
317      ctrl.Rule(temp["high"], severity_out["severe"]),  
318      ctrl.Rule(torque["high"], severity_out["severe"]),  
319      ctrl.Rule(rpm["high"], severity_out["severe"]),  
320      ctrl.Rule(delay["medium"] & temp["medium"], severity_out["severe"]),  
321      ctrl.Rule(delay["medium"] & rpm["medium"], severity_out["severe"]),  
322      ctrl.Rule(temp["medium"] & torque["medium"], severity_out["severe"]),  
323      ctrl.Rule(torque["medium"] & rpm["medium"], severity_out["severe"]),  
324  ]
```

Figure 33 - Fuzzy Logic Rules

5.5.3 Results

The Fuzzy Logic system achieved 41.67% severity accuracy and 58.33% action accuracy, with an average inference time of 31.380 ms for the batch, or 2.6150 ms per record. This was a slow approach. A major weakness appeared in severity prediction: the model failed to correctly identify any Normal or Moderate samples and classified all Severe cases correctly, which indicates a strong bias toward predicting higher severity. In the severity confusion matrix, all five actual moderate samples were predicted as severe, and both actual normal samples were predicted as moderate. Although the action accuracy was moderate, this was not enough to compensate for the weak severity classification performance.

```
Inference time (average for 12 test records): 31.380 ms  
Per-record inference time: 2.6150 ms  
Severity accuracy: 41.67%  
Action accuracy: 58.33%
```

Figure 34 - Fuzzy Logic overall results

-- Severity Classification Report (test set) --				
	precision	recall	f1-score	support
Normal	0.00	0.00	0.00	2
Moderate	0.00	0.00	0.00	5
Severe	0.50	1.00	0.67	5
accuracy			0.42	12
macro avg	0.17	0.33	0.22	12
weighted avg	0.21	0.42	0.28	12

Severity Confusion Matrix			
	Pred_Normal	Pred_Moderate	Pred_Severe
Actual_Normal	0	2	0
Actual_Moderate	0	0	5
Actual_Severe	0	0	5

Severity FP/FN Summary				
class	TP	FP	FN	TN
Normal	0	0	2	10
Moderate	0	2	5	5
Severe	5	5	0	2

Figure 35 - Fuzzy Logic Severity Classification Report, severity confusion matrix and FP/FN summary

-- Action Classification Report (test set) --				
	precision	recall	f1-score	support
NO_ACTION	0.00	0.00	0.00	2
SHIFT_TIMING_ADJUST	0.40	1.00	0.57	2
CLUTCH_PRESSURE_RECALIB	1.00	1.00	1.00	1
TORQUE_REDISTRIBUTION	0.75	0.50	0.60	6
RPM_SYNC	0.50	1.00	0.67	1
accuracy			0.58	12
macro avg	0.53	0.70	0.57	12
weighted avg	0.57	0.58	0.53	12

Action Confusion Matrix					
	Pred_NO_ACTION	Pred_SHIFT_TIMING_ADJUST	Pred_CLUTCH_PRESSURE_RECALIB	Pred_TORQUE_REDISTRIBUTION	Pred_RPM_SYNC
Actual_NO_ACTION	0	1	0	1	0
Actual_SHIFT_TIMING_ADJUST	0	2	0	0	0
Actual_CLUTCH_PRESSURE_RECALIB	0	0	1	0	0
Actual_TORQUE_REDISTRIBUTION	0	2	0	3	1
Actual_RPM_SYNC	0	0	0	0	1

Action FP/FN Summary				
class	TP	FP	FN	TN
NO_ACTION	0	0	2	10
SHIFT_TIMING_ADJUST	2	3	0	7
CLUTCH_PRESSURE_RECALIB	1	0	0	11
TORQUE_REDISTRIBUTION	3	1	3	5
RPM_SYNC	1	1	0	10

Figure 36 - Fuzzy Logic Action Classification Report, Action confusion matrix and FP/FN summary

5.6 Rule-Based Approach

5.6.1 Threshold Design

The Rule-Based system used thresholds derived from the training data. For each sensor feature, two thresholds were calibrated: one representing the boundary from Normal to Moderate and another representing the boundary from Moderate to Severe. These thresholds were estimated using class medians and adjusted when necessary to maintain a valid ordering between the two boundaries.

5.6.2 Decision Logic

The rule engine then counted how many features exceeded the moderate threshold and how many exceeded the severe threshold. If at least two features crossed the severe threshold, or if one feature was severe while at least three were moderate or above, the anomaly was classified as Severe. If at least one feature crossed the moderate threshold, the anomaly was classified as Moderate. Otherwise, the condition was classified as Normal. The final correction action was selected using the dominant abnormal feature in the same way as the fuzzy system.

```
369     for idx, feature in enumerate(FEATURES):
370         value = float(row[idx])
371         if value >= thresholds[feature]["moderate_to_severe"]:
372             severe_hits += 1
373         if value >= thresholds[feature]["normal_to_moderate"]:
374             moderate_hits += 1
375
376         if severe_hits >= 2 or (severe_hits >= 1 and moderate_hits >= 3):
377             severity = "Severe"
378         elif moderate_hits >= 1:
379             severity = "Moderate"
380         else:
381             severity = "Normal"
```

Figure 37 - Rule-based Decision Logics

5.6.3 Results

The Rule-Based approach achieved 50.00% severity accuracy and 58.33% action accuracy, with an average inference time of only 0.027 ms for the full batch, or 0.0022 ms per record. This is a faster approach. The severity results show that the method detected all five Severe cases correctly, but it failed to correctly classify any Normal cases and correctly classified only one of the five Moderate cases. In practice, this means the system was efficient but overly simplistic, with a tendency to push borderline cases toward higher severity. Therefore, although the rule-based approach is attractive for real-time speed, its anomaly detection effectiveness is weaker.

```
Inference time (average for 12 test records): 0.027 ms
Per-record inference time: 0.0022 ms
Severity accuracy: 50.00%
Action accuracy: 58.33%
```

Figure 38 - Rule-based overall results

-- Severity Classification Report (test set) --				
	precision	recall	f1-score	support
Normal	0.00	0.00	0.00	2
Moderate	0.33	0.20	0.25	5
Severe	0.56	1.00	0.71	5
accuracy			0.50	12
macro avg	0.30	0.40	0.32	12
weighted avg	0.37	0.50	0.40	12
Severity Confusion Matrix				
	Pred_Normal	Pred_Moderate	Pred_Severe	
Actual_Normal	0	2	0	
Actual_Moderate	0	1	4	
Actual_Severe	0	0	5	
Severity FP/FN Summary				
class	TP	FP	FN	TN
Normal	0	0	2	10
Moderate	1	2	4	5
Severe	5	4	0	3

Figure 39 - Rule-based Severity Classification Report, severity confusion matrix and FP/FN summary

-- Action Classification Report (test set) --					
	precision	recall	f1-score	support	
NO_ACTION	0.00	0.00	0.00	2	
SHIFT_TIMING_ADJUST	0.40	1.00	0.57	2	
CLUTCH_PRESSURE_RECALIB	1.00	1.00	1.00	1	
TORQUE_REDISTRIBUTION	0.75	0.50	0.60	6	
RPM_SYNC	0.50	1.00	0.67	1	
accuracy				0.58	12
macro avg				0.57	12
weighted avg				0.53	12
Action Confusion Matrix					
	Pred_NO_ACTION	Pred_SHIFT_TIMING_ADJUST	Pred_CLUTCH_PRESSURE_RECALIB	Pred_TORQUE_REDISTRIBUTION	Pred_RPM_SYNC
Actual_NO_ACTION	0	1	0	1	0
Actual_SHIFT_TIMING_ADJUST	0	2	0	0	0
Actual_CLUTCH_PRESSURE_RECALIB	0	0	1	0	0
Actual_TORQUE_REDISTRIBUTION	0	2	0	3	1
Actual_RPM_SYNC	0	0	0	0	1
Action FP/FN Summary					
class	TP	FP	FN	TN	
NO_ACTION	0	0	2	10	
SHIFT_TIMING_ADJUST	2	3	0	7	
CLUTCH_PRESSURE_RECALIB	1	0	0	11	
TORQUE_REDISTRIBUTION	3	1	3	5	
RPM_SYNC	1	1	0	10	

Figure 40 - Rule-based Action Classification Report, Action confusion matrix and FP/FN summary

5.7 Comparative Evaluation of the Three Approaches

A fair comparison summary was created using severity accuracy, action accuracy, severity F1, action F1, inference time and a combined suitability score. The results show that the Machine Learning approach ranked first with a suitability score of 0.8402, followed by the Rule-Based system with 0.5212, and finally Fuzzy Logic with 0.3654. Machine Learning was not the fastest method, but it clearly produced the highest predictive performance. The Rule-Based approach was extremely fast, but its classification quality was much weaker. Fuzzy Logic was the weakest overall because it was both the slowest and the least accurate in severity detection. From these results, it can be seen that high speed alone is not sufficient for AMT deployment if anomaly detection quality drops too much.

FAIR COMPARISON SUMMARY							
Approach	Severity Accuracy (%)	Action Accuracy (%)	Severity F1	Action F1	Inference Time (ms)	Suitability Score	
Machine Learning	75.00	91.67	0.7489	0.9212	3.5818	0.8402	
Rule-Based	50.00	58.33	0.4018	0.5341	0.0270	0.5212	
Fuzzy Logic	41.67	58.33	0.2778	0.5341	31.3797	0.3654	
Recommended approach for real-time AMT deployment:							
Machine Learning Suitability Score = 0.8402 Inference Time = 3.5818 ms							

Figure 41 - Overall Comparative Evaluation

5.8 Best Approach Selection and Justification

Based on the experimental results, the **Machine Learning approach is the most suitable method** for real-time AMT deployment. This decision is justified by considering both inference speed and anomaly mitigation effectiveness. Although the Rule-Based method is much faster, its lower severity accuracy and weak performance on normal and moderate cases make it less reliable for real transmission monitoring. The Fuzzy Logic model also underperformed, showing the slowest response time and the weakest severity classification. In contrast, Machine Learning achieved the highest severity and action accuracy, the highest F1 scores and still maintained a very low per-record inference time of approximately 0.2985 ms, which is small enough for practical real-time use. Therefore, it provides the best trade-off between responsiveness and corrective reliability.

6.AI Usage Declaration

For Question 1, AI assistance was used to better understand how a Constraint Satisfaction Problem should be formulated, including decision variables, constraints, objective function structure and how these could be explained clearly in report form.

For Question 2, AI assistance was used to discuss possible ontology scope, improve wording for the competency questions and refine the written explanations of RDF/OWL design and SPARQL query results.

For Question 3, AI assistance was used to clarify the theoretical differences between Iterative Deepening Depth-First Search and Best First Search and to improve the explanation of heuristics, performance analysis, and discussion writing.

For Question 4, AI assistance was used to discuss suitable ways of presenting the Machine Learning, Fuzzy Logic and Rule-Based approaches, improve explanation of the results and refine the comparative evaluation section.

7. Appendix

Resource codes are also submitted. The relevant file names for each question are given below.

Q1. Question1.xlsx

Q2. Question2Automotive.rdf

Q3. Question3.py

Q4. Question4.py

8. Reference

Microsoft (n.d.) *Using Solver to schedule your workforce*. Microsoft Support. [Online]. Available at: <https://support.microsoft.com/en-gb/office/using-solver-to-schedule-your-workforce-7ae8dd69-7582-4f41-83f1-ea3543398b46> (Accessed: 11 April 2026).

National Highway Traffic Safety Administration (n.d.) *NHTSA Statutes, Regulations, Authorities & FMVSS*. [online] Available at: <https://www.nhtsa.gov/laws-regulations> (Accessed: 11 April 2026).

AIAG (n.d.) *Quality Core Tools*. [online] Available at: <https://www.aiag.org/expertise-areas/quality/quality-core-tools> (Accessed: 13 April 2026).

United Nations Economic Commission for Europe (2021) *UN Regulation No. 155 – Cyber security and cyber security management system*. [online] Available at: <https://unece.org/transport/documents/2021/03/standards/un-regulation-no-155-cyber-security-and-cyber-security> (Accessed: 13 April 2026).

Monolithic Power Systems (n.d.) *Basics of Automotive Electronic Systems*. [online] Available at: <https://www.monolithicpower.com/en/learning/mpscholar/automotive-electronics/introduction/basics-of-automotive-electronic-systems> (Accessed: 14 April 2026).

Pendar, M.-R., Rodrigues, F., Páscoa, J.C. and Lima, R. (2022) ‘Review of coating and curing processes: Evaluation in automotive industry’, *Physics of Fluids*, 34(10), p. 101301. [online] Available at: <https://doi.org/10.1063/5.0109376> (Accessed: 14 April 2026).

Hossain, M.N., Rahman, M.M. and Ramasamy, D. (2024) ‘Artificial Intelligence-Driven Vehicle Fault Diagnosis to Revolutionize Automotive Maintenance: A Review’, *CMES - Computer Modeling in Engineering and Sciences*, 141(2), pp. 951–996. [Online]. Available at: <https://doi.org/10.32604/cmes.2024.056022> (Accessed: 15 April 2026).